The Exploitability Gap in LLM-Serving Scheduler Portfolios

Soroush Vahidi[1]

New Jersey Institute of Technology, Newark, NJ, USA
sv96@njit.edu

Abstract. LLM-serving schedulers have complementary strengths, motivating portfolios. On a 240-scenario joint workload, per-scenario best-policy choice improves arrival-normalized weighted goodput (ANWG) by 0.0190 over the best fixed policy (SBS). On the same held-out scenarios, two causal adaptive methods—a scenario-level selector and a live online router—both perform below SBS (realized gain −0.008 and −0.030; 95% CIs exclude zero), so this opportunity is not recovered by the tested architectures. A terminal counterfactual analysis explains part of why: consequential scheduling decisions are rare and concentrated, and native policy disagreement is a near-chance predictor of them. Real-vLLM validation separately shows simulator–engine semantic mismatch and a native token-budget tradeoff.

Keywords: LLM serving · scheduling · portfolios · vLLM

1 Introduction

LLM inference serving has to schedule heterogeneous work. Requests differ in prompt length, predicted and realized generation length, SLO or deadline tightness, class or tenant weight, KV-cache footprint, burst timing, and interactions between prefill and decode. These dimensions stress different mechanisms in the serving stack: one workload may reward uninterrupted prompt processing, another may reward urgent-deadline protection, and another may reward preserving scarce KV capacity. It is therefore unlikely that one simple scheduling rule will dominate all operating regimes.

This makes scheduler portfolios appealing, echoing algorithm-selection work that compares a single best solver against a virtual best solver (SBS versus VBS) [26, 8, 37, 17]. The VBS measures *scenario-level policy complementarity* as offline opportunity; an online adaptive scheduler must act from causal observations under closed-loop queue dynamics. Whether that opportunity is recoverable is a distinct *exploitability* question, formalized in Section 2.4.

Even when regimes are detectable, a router may fail to recover utility because high-value intervals are sparse, closed-loop execution shifts decision support, or coarse labels misalign with action-value boundaries. We therefore ask not only whether policies complement one another, but whether tested adaptive mechanisms can exploit that structure under causal serving information.

We study a fixed six-policy portfolio motivated by LLM-serving mechanisms such as iteration-level scheduling, PagedAttention, and chunked prefill [38, 16, 1], together with disaggregated and fairness-oriented serving ideas [39, 31]. The portfolio spans full and chunked prefill, estimated-service ordering, weighted fair sharing, least-laxity urgency, and KV-aware admission. We evaluate complementarity across controlled and joint workloads and local vLLM, then test adaptive exploitation, terminal-criticality analysis, and constructive alternatives under preregistered splits, detailed in the four contributions below. Except where noted, results are interpreted across separately scoped settings (Section 2.4); they do not constitute one shared numeric decomposition of joint-workload headroom.

This paper makes four contributions:

1. We show workload-dependent complementarity and positive VBS headroom for a six-policy portfolio, under jointly varying pressures and after expanding the envelope with independently motivated external baselines.
2. On the same joint workload used to measure that headroom, we compare SBS, VBS, a scenario-level causal selector, and a live causal router: both adaptive methods fall below SBS, with CIs excluding zero improvement, so measured opportunity is not automatically exploitable by the tested architectures.
3. Using a causal terminal-utility fork-and-continue methodology, we show consequential scheduling decisions are sparse and concentrated, and that native disagreement and an earlier short-horizon proxy both fail to predict them—one explanation for the underperformance above.
4. We test constructive exploitation beyond selection (support expansion, guarded composition, typed synthesis) and validate a simulator-derived mechanism against native vLLM; results remain bounded negative, or reveal a semantic mismatch and a native token-budget tradeoff.

2 Portfolio, Workloads, and Metrics

2.1 Problem setting

We model LLM-serving as an online scheduling problem over arriving requests, each with an arrival time, prompt-token length, predicted generation length, class/tenant identifier, priority weight, and SLO deadline. During execution the simulator exposes causal state—queued and admitted/running requests, elapsed waiting time, remaining or predicted service proxies, active sequence counts, prefill/decode phase, and KV capacity/pressure—and a policy chooses which request to admit or serve next, how prompt work is chunked when that control exists, and whether KV/load constraints allow admission.

The policy input is intentionally online-observable. The future actual output length of an unfinished request is not exposed; it is used only after a run to compute outcomes. Thus offline VBS quantities, including the best policy for a scenario, are analytical baselines rather than scheduler inputs.

2.2 Six-policy portfolio

The fixed portfolio contains six policies spanning distinct mechanisms. Table 1 summarizes the paper-facing interpretation; implementation details remain in the repository artifacts.

Table 1. Six-policy portfolio used for envelope and complementarity analysis. Online state is restricted to quantities observable during serving; future actual output length is never an input.

Policy	Mechanism	Key online state
Full prefill	Uninterrupted prompt processing for long prompt-heavy work	Queue order, prompt phase, prefill budget
Small-chunk prefill	Prompt chunking for prefill/decode interleaving and late TTFT	Queue order, prompt phase, chunk budget
ESTF	Short estimated-service first for completion efficiency	Prompt tokens, predicted output tokens
WFS	Weighted service-deficit fairness and SLO protection	Class weights, admitted/completed service
LLF	Least-laxity urgency under deadlines	Deadline, current time, predicted service
KV constrained	KV-reserve admission/scheduling with urgent bypass	KV capacity, estimated KV demand, laxity

These are repository-specific baselines, not full reproductions. `full_prefill` follows iteration-level batching in Orca/vLLM [38, 16]. `chunked_prefill_small` is motivated by chunked-prefill work such as Sarathi-Serve [1] (our simulator omits its full stall-free decode-priority algorithm). ESTF follows SPT/SRPT intuition [30]; WFS follows GPS/WFQ-style fairness [22, 6, 31]; LLF is a real-time urgency heuristic; KV constrained relates to KV-aware serving [16, 12].

2.3 Workload suites

We use four workload categories. First, public trace replay provides a realism baseline using BurstGPT and Azure 2023 LLM-inference traces [33, 20, 23]. We construct 20 200-request windows from each of BurstGPT, Azure conversation, and Azure code, retaining source order, relative timestamps, and prompt/output token counts; predicted lengths, deadlines, priorities, and classes are deterministic overlays, and actual future output length is hidden from online policies. This configuration motivates stress workloads but does not show public traces are generally uninformative.

Second, controlled Families A/B/C isolate mechanisms. Family A stresses fairness, completion, and SLO conflict; Family B stresses prefill/decode contention; and Family C stresses KV pressure and urgency. These families are the original controlled evidence for complementarity and later selector/router/synthesis tests.

Third, the joint multi-mechanism workload broadens the evidence beyond concatenated stress families. It contains 240 scenarios drawn from one common schema that jointly varies offered load, burstiness, long-vs-short mixture, prompt-length heterogeneity, output/service-length heterogeneity, tenant weight skew, SLO tightness, prediction noise, KV pressure, late-arrival structure, active-sequence capacity, and step-token budget. Its pressure audit is outcome-independent: it computes six normalized indicators in $[0, 1]$ covering fairness, service heterogeneity, prefill/decode contention, KV pressure, urgency, and burst pressure. A pressure is counted as elevated when the corresponding indicator is at least 0.60, as specified before policy evaluation. Under this audit, 223 of 240 scenarios (92.9%) have at least two elevated mechanism pressures, and 175 have at least three.

Fourth, local real-vLLM validation on Qwen2.5-0.5B served by vLLM 0.27.1 on an RTX 5060 Ti tests whether the simulator's prefill/decode abstraction corresponds to native behavior and whether native vLLM exposes its own scheduling-control tradeoff; these experiments are not used to tune the simulator.

Across these studies, practical-effect and robustness criteria were specified before held-out or screening evaluations; selector/router analyses distinguish offline labels from live closed-loop re-evaluation. Joint-workload CIs use 1,000 scenario-bootstrap resamples; native-vLLM CIs use percentile bootstraps over five paired repetition-level differences after matched warmups and ABBA ordering.

2.4 Metric and envelope definitions

The primary utility is arrival-normalized weighted goodput (ANWG):

$$\mathrm{ANWG} = \frac{\sum_i w_i \mathbf{1}[i \text{ completed}]\mathbf{1}[C_i \le D_i]}{\sum_i w_i}. \tag{1}$$

Dropped, unfinished, or late completions receive zero numerator credit but remain in the arrival-weight denominator.

For evaluation set X, portfolio P (here P_6), policy utility $R_h(x)$, and adaptive utility $R_A(x)$, the single best scheduler (SBS) and virtual best scheduler (VBS) are

$$h^* = \arg\max_{h \in P} \frac{1}{|X|} \sum_{x \in X} R_h(x), \quad R_{\mathrm{SBS}}(X, P) = \frac{1}{|X|} \sum_{x \in X} R_{h^*}(x), \tag{2}$$

$$E_P(x) = \max_{h \in P} R_h(x), \quad R_{\mathrm{VBS}}(X, P) = \frac{1}{|X|} \sum_{x \in X} E_P(x). \tag{3}$$

We write $E_6(x)$ when $P = P_6$. An ϵ-unique winner satisfies $R_h(x) \ge \max_{q \ne h} R_q(x) + \epsilon$ ($\epsilon = 0.01$ ANWG). VBS headroom is

$$\mathrm{Headroom}(X, P) = R_{\mathrm{VBS}}(X, P) - R_{\mathrm{SBS}}(X, P). \tag{4}$$

VBS headroom measures portfolio opportunity; the exploitability gap is defined relative to a particular adaptive mechanism on the same X. Realized gain is $R_A(X) - R_{\mathrm{SBS}}(X, P)$; selector regret is mean $E_P(x) - R_{\hat{h}(x)}(x)$ for selected parent $\hat{h}(x) \in P$. The exploitability gap is

$$\mathrm{EG}(A; X, P) = R_{\mathrm{VBS}}(X, P) - R_A(X) = \frac{1}{|X|} \sum_{x \in X} (E_P(x) - R_A(x)). \tag{5}$$

Each headroom or gap metric is scoped to its own X; equating values across datasets is invalid unless evaluations share X and P. When $\mathrm{Headroom}(X, P) > 0$, normalized VBS-gap closure is

$$\mathrm{GapClosure}(A; X, P) = \frac{R_A(X) - R_{\mathrm{SBS}}(X, P)}{R_{\mathrm{VBS}}(X, P) - R_{\mathrm{SBS}}(X, P)}, \tag{6}$$

with residual $\mathrm{NEG}(A; X, P) = 1 - \mathrm{GapClosure}(A; X, P)$. Zero headroom makes closure undefined; values below 0 or above 1 indicate underperformance or out-of-envelope behavior and require cautious interpretation. Synthesized marginal envelope gain averages $\max(R_c(x), E_P(x)) - E_P(x)$ over the pre-specified training screen.

3 Portfolio Complementarity

An adaptive portfolio requires useful parent disagreement. We ask whether policies produce workload-dependent winners under controlled mechanisms and whether that complementarity persists under jointly varying pressures.

3.1 Controlled mechanism families

The controlled A/B/C suites isolate specific serving tensions. Family A varies fairness, completion, and SLO conflict; Family B varies prompt-heavy prefill pressure against late short requests; and Family C varies KV pressure and urgent arrivals. Across the unified 176-scenario, six-policy matrix, the portfolio produces 1,056 policy cells and a 54.0% ϵ-unique-winner rate. The best fixed policy and VBS headroom differ by family: WFS is best fixed in Family A with 0.021742 headroom, full prefill is best fixed in Family B with 0.049433 headroom, and `kv_constrained_online` is best fixed in Family C with 0.020261 headroom; these results isolate mechanisms, not a production-traffic claim.

3.2 Public-trace saturation

Public-trace replay is a sanity check, not the main discriminating benchmark. Under the processed replay configuration, all 60 windows tie at ANWG 1.0, six-policy envelope gain is 0.0, active count p99 is 5 out of 512, and maximum KV utilization is 0.003802: this configuration does not expose scheduler differentiation for the current objective.

3.3 Joint multi-mechanism workload

To test whether complementarity is merely an artifact of disjoint handcrafted families, we generated a 240-scenario joint workload from one common schema before evaluating policy outcomes (Figure 1). Per-scenario winners remain distributed across the portfolio: LLF wins 59 scenarios, `kv_constrained_online`

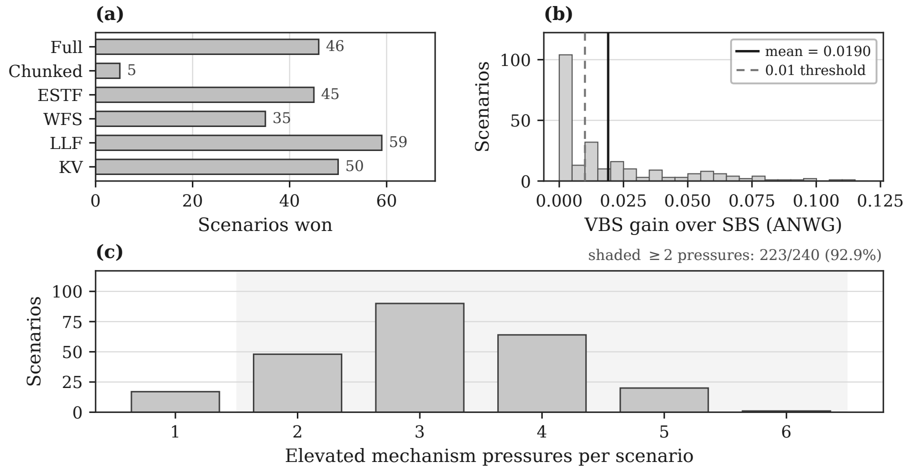

Fig. 1. Joint 240-scenario multi-mechanism workload under P_6 (excludes the external expansion, Section 3.4). (a) Per-scenario VBS winner counts. (b) Per-scenario VBS gain over SBS (ANWG); solid = mean, dashed = 0.01. (c) Elevated mechanism pressures per scenario (threshold 0.60); shading marks ≥ 2 pressures. Winner diversity and positive headroom persist under jointly varying pressures.

50, full prefill 46, ESTF 45, WFS 35, and small-chunk prefill 5. The ϵ-unique-winner fraction is 59.6%, nontrivial policy spread appears in 91.7% of scenarios, and the mean policy range is 0.111658 ANWG.

The SBS over the joint workload is `kv_constrained_online`, with mean ANWG 0.314072. The six-policy VBS reaches mean ANWG 0.333106, yielding VBS headroom 0.019034 with bootstrap 95% CI [0.015988, 0.022433]. The median per-scenario VBS gain is 0.010622, p90 gain is 0.058040, 60.4% of scenarios have positive gain, and 51.3% have gain at least 0.01. Most gain arises outside single-mechanism extremes: 93.1% comes from scenarios with at least two elevated pressures and 63.3% from scenarios with at least three; the top 10% of scenarios account for 40.5% of total gain: bounded synthetic evidence, not a production claim.

3.4 External-portfolio robustness

A natural concern is that P_6 opportunity is an artifact of weak baselines. We expand the portfolio on the same joint-240 scenarios to $P_{\text{ext}} = P_6 \cup \{\text{VTC}, \text{vLLM-style}\}$, using an official VTC implementation [31] and a disclosed continuous-batching proxy. Learning-to-rank and SOLA-style systems remain separate evidence [7, 10]; they are not members of P_{ext}. Adaptive experiments in Sections 4–5 remain scoped to P_6 and were not retroactively retrained over eight policies.

Table 2 summarizes the envelope comparison. SBS is unchanged. VBS rises only by +0.000214 (bootstrap CI $[0.00004, 0.00046]$), essentially the VTC in-

Table 2. Joint-240 portfolio envelopes for P_6 and $P_{\text{ext}} = P_6 + \text{VTC} + \text{vLLM-style}$. SBS policy is `kv_constrained_online` in both cases. Bootstrap 95% CIs use 1,000 scenario resamples (seed 20260825).

Portfolio	SBS	VBS	Headroom	ΔVBS vs. P_6
P_6	0.314072	0.333106	0.019034	—
P_{ext}	0.314072	0.333319	0.019247	+0.000214

cremental envelope alone; the vLLM-style proxy adds ≈ 0. Headroom remains ≈ 0.019 (Pext CI $[0.0162, 0.0226]$). Under P_{ext}, winners remain diverse (LLF 57, KV 48, ESTF 45, WFS 34, full prefill 27, VTC 23, chunked 3, vLLM-style 3), and leave-one-out removal of any P_6 policy still reduces VBS (largest drops: KV ≈ 0.0071, LLF ≈ 0.0058). Thus strong external baselines do not collapse the observed opportunity: they neither replace the SBS nor erase P_6 unique envelope contribution. This is a portfolio-scope robustness test, not a claim of superiority over production VTC or native vLLM.

3.5 From opportunity to exploitability

Section 3 establishes positive VBS headroom, including on joint-240. Section 4 first summarizes two earlier adaptive tests on other scopes, then directly compares SBS, VBS, and two causal adaptive methods on the *same* joint-240 held-out scenarios, and finally uses a causal terminal-utility analysis to help explain why recovery is difficult (Section 2.4).

4 Adaptive Exploitation

We next evaluate whether tested adaptive mechanisms can convert complementarity into deployable utility under preregistered criteria and evaluation scopes (Section 2.4).

4.1 Earlier controlled adaptive tests

On the unified 176-scenario controlled-family matrix, within-family holdouts showed learnability (Families A/B at zero regret; Family C competitive), but pooled cross-family selection failed: best pooled regret 0.0463 exceeded best-fixed 0.0233 and majority 0.0127, and leave-one-family-out transfer was unstable (Family-A held-out regret 0.4786 versus fixed 0.0767). A shared-feature audit found perfect family classifiability but no utility-consistent cross-family neighbors, so this rescue did not yield a robust cross-family selector.

Separately, a hierarchical router achieved macro-F1 0.9887 with no catastrophic misrouting, yet on live closed-loop re-evaluation gained only +0.00616 mean ANWG over the best global fixed policy (below the +0.01 criterion) and achieved VBS-gap closure 0.143 (Eq. 6), far below the 0.75 target—an instance of

Eq. 5: accurate regime detection can coexist with large residual opportunity because labels are coarser than action-value boundaries and closed-loop scheduling shifts the state distribution under which specialized policies were valuable.

Table 3. Cross-experiment summary of pre-specified adaptive-exploitation criteria. Each row uses its own evaluation scope and is not comparable as a shared-dataset leaderboard.

Method	Scope	Intermediate signal	Result
Pooled selector	A/B/C (176)	Within-family learnability	Regret 0.0463 vs. fixed 0.0233 / maj. 0.0127
Hierarchical router	Live	Macro-F1 0.9887; 0 catastrophic	+0.00616 ANWG (< 0.01); GapClosure 0.143
Terminal-ANWG fork	TRAIN/VAL (144)	27/734 (3.7%) nonzero; disagr. AUROC 0.513	Top-1% mass 48.3%; H10-proxy Spearman 0.005
Support expansion	Family-A	24,314 fingerprints; 156 domains	1/104 closer; p90 = 0; 0/8 criteria
Guarded composition	Family-A	WFS regret reduction 3.11%	< 10% min.; quadrant order fail
Typed crossover	24 TRAIN	Exact parents; 4,320 evals	Best marginal gain 0.0; no promotion

Table 3 summarizes bounded negative evidence across settings, including the same-distribution and terminal-criticality results detailed next; it does not imply that all adaptive schedulers must fail.

4.2 Same-distribution joint-240 test

Section 3 measures VBS headroom offline; here we test whether it is recoverable by causal adaptive methods evaluated on the *same* joint-240 scenarios, using preregistered 5-fold out-of-fold cross-fitting (seed 20260825). We restrict to P_6 and test two causal architectures: a scenario-level selector (A_{scen}, logistic regression on 17 pre-policy features, scored via a frozen utility lookup) and a live 6-way router (A_{live}, retrained online features, dwell ≥ 20 steps per policy before switching, scored by closed-loop simulation); live-vs-matrix agreement was verified to 4×10^{-7}.

Both methods perform *below* SBS, not merely below VBS: gain is negative for both (95% CIs exclude zero), and catastrophic regressions ($R_A < R_{\text{SBS}} - 0.01$) affect 28% of A_{scen} scenarios and 49% of A_{live}'s. A_{scen}'s winner classification is modestly above chance (accuracy 29.6%, macro-F1 0.267 vs. 16.7% chance), but mean regret is 0 when correct and 0.0386 when not: modest classification accuracy is insufficient because misclassification consequences are highly nonuniform. On the same workload used to measure P_6 opportunity, neither adaptive method recovers it.

Table 4. Same-distribution joint-240 results ($n = 240$ out-of-fold scenarios; headroom 0.019034; paired bootstrap, $B = 1{,}000$, seed 20260825). SBS policy is `kv_constrained_online`. Gain and GapClosure are relative to SBS; a negative GapClosure means $R_A < R_{\mathrm{SBS}}$, i.e. added regret, not partial recovery.

Method	ANWG	Gain [95% CI]	GapClosure	Catastrophic
SBS	0.3141	—	—	—
VBS	0.3331	—	—	—
Majority	0.2909	−0.023 [−0.031, −0.016]	−1.22	109/240
A_{scen}	0.3059	−0.008 [−0.013, −0.003]	−0.43	67/240
A_{live}	0.2840	−0.030 [−0.038, −0.023]	−1.58	118/240

4.3 Why adaptation fails: terminal decision criticality

To help explain Section 4.2, we ran a separate causal counterfactual study on the TRAIN/VAL controlled corpus (Families A/B/C, 144 scenarios, distinct from joint-240). At up to 5 disagreement and 3 agreement-control points per scenario (734 evaluated states, 124 scenarios), we forked the live simulator, forced one alternative P_6 policy's action, continued via a cloned live router to natural termination, and measured the terminal effect $C_t = \mathrm{ANWG}_{\mathrm{CF}} - \mathrm{ANWG}_{\mathrm{ref}}$; 124/124 reference-replays matched the untouched trajectory to 10^{-12}.

Consequential decisions are rare and sharply concentrated: 96.3% of evaluated states have exactly zero terminal effect. Of the 3.7% (27/734) that are nonzero, the top 1% of *all* evaluated states already carry 48.3% of total $|\Delta\mathrm{ANWG}|$ mass, and just 2 of 144 scenarios carry 42.4% of it. Effects are enriched in prefill/decode contention (Family B, 13.5% nonzero) relative to fairness (Family A, 1.4%) and KV pressure (Family C, 3.1%). Two cheaper proxies both fail: native policy disagreement is a near-chance predictor of nonzero terminal effect (AUROC 0.513, enrichment 1.11×), and an earlier short-horizon completed-count proxy is essentially uncorrelated with it (Spearman 0.005; 92.9% of terminal-critical states are missed by that proxy). When an intervention does have nonzero terminal effect, it induces sustained downstream trajectory divergence in 100% of cases (versus 33.5% among zero-effect states), with a median 2,329-step post-fork horizon before completion—closed-loop consequences persist even though the triggering decision is usually an isolated single step. This helps explain why disagreement- or short-horizon-driven signals are poor guides in Section 4.2, though this corpus is TRAIN/VAL A/B/C, not joint-240, and its 27 nonzero observations do not by themselves establish state-level criticality there.

5 Constructive Exploitation Beyond Selection

We next test stronger exploitation mechanisms beyond parent selection, each under pre-specified criteria (Table 3). Target-free **support expansion** added 24,314 behavioral fingerprints (encoded state-action traces) spanning 156 training-side domains (state-space regions) without using held-out Family-A states as targets, but moved only 1/104 held-out states closer under the primary nearest-neighbor metric (p90 improvement 0), rejecting insufficient training-side support as a sufficient explanation in this formulation. Family A showed ESTF/WFS complementarity, so we tested **guarded semantic composition**: the best WFS-default composite rule with explicit ESTF release guards reduced WFS regret by only 3.11% (below the 10% minimum-effect criterion) and violated the required completion-versus-SLO quadrant ordering. Finally, a **typed program synthesis** screen with exact parent reproduction compared random search, mutation-only search, and portfolio-seeded structural crossover (24 training scenarios, 60 candidates/treatment, 4,320 evals); best mean marginal gains were 0.011295 (random), 0.002551 (mutation), and 0.0 (crossover), with no candidate meeting promotion criteria.

6 Real-System Validation

On this local probe (Section 2) we validated prefill/decode contention, testing qualitative mechanism transfer, not exact magnitude matching.

6.1 Direct Family-B analogue

Family B fixes a 512-token simulator step budget and varies prefill chunk size (full 65,536 vs. small-chunk 64). The native analogue varied chunking and the native token budget (4096 vs. 512). All 300/300 requests completed with real queueing, but native chunking did not reproduce the simulator-like late-TTFT reversal under pre-specified criteria.

6.2 Semantic diagnosis

Diagnosis revealed semantic mismatch: native vLLM V1 does not implement the same fixed-budget FCFS abstraction, and the native scheduler token budget is itself a control variable. Scheduler traces overlapped but were not equivalent (FULL: 6 mixed prefill+decode steps, 0 partial chunks; CHUNKED: 16 mixed steps, 21 partial chunks). Prefill remained nontrivial (3,200-token prefill $\approx$0.0767 s vs. 0.00591 s/token decode).

6.3 Native token-budget control

With chunked prefill enabled and all other controls fixed, varying only the native token budget (512 vs. 4096) produced repeatable workload-dependent latency effects (Figure 2): two regimes, 20 measured regime-runs, 150/150 successful requests, ABBA ordering.

In the low-late regime, T4096 improved late-request TTFT by 30.6 ms (95% CI [-38.0, -22.8] ms; 5/5 sign consistency) but worsened prompt-heavy E2E by 16.3 ms (CI [9.2, 23.5] ms). In the high-late regime, late-TTFT change was -2.5 ms (CI [-7.7, 1.8] ms; 3/5 consistency) while prompt-heavy E2E worsened by 23.3 ms (CI [15.6, 31.8] ms). This is a native token-budget tradeoff in the tested configuration, not a Family-B reproduction [16, 1, 39, 25].

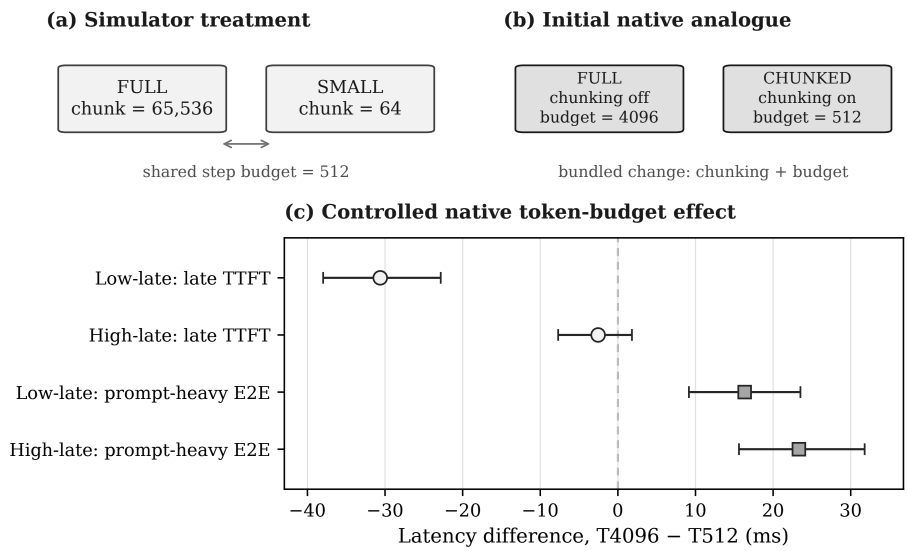

Fig. 2. Simulator-to-native vLLM validation on the local probe. (a) Simulator treatment: shared 512-token step budget, only prefill chunk size varies. (b) Initial native analogue: chunking and token budget change together (bundled, not single-factor). (c) Controlled native experiment, chunked prefill fixed; points are T4096−T512 latency differences with 95% bootstrap CIs. Negative late-request TTFT favors T4096; positive prompt-heavy E2E favors T512.

7 Related Work, Limitations, and Conclusion

Related work. Many LLM-serving systems already adapt online: migration (Llumnix [32]), queueing control (QLM [24]), state-aware/multi-SLO scheduling (SOLA, SLOs-Serve [10, 4]), hierarchical multi-timescale routing (H-MAS [18]), flexible request partitioning (Libra [29]), and online policy self-evolution (Autopoiesis [13]). Others target one mechanism: batching/memory (Orca, vLLM [38, 16]); chunked prefill and prefill/decode isolation (Sarathi-Serve, DistServe, Splitwise, TetriInfer [1, 39, 23, 11]); KV-centric serving (Mooncake [25]); fairness via virtual tokens (VTC [31]); preemption (FastServe [35]); rank ordering [7]; goodput/phase-aware scheduling (Past-Future, PASCAL [9, 5]); and KV-constrained formulations [12, 2]. To our knowledge, none jointly quantifies SBS/VBS portfolio headroom, evaluates causal adaptive selectors against that opportunity on the same workload distribution, and analyzes terminal counterfactual scheduling criticality—our focus. H-MAS is closest for hierarchical adaptive routing but omits portfolio opportunity and terminal criticality; Autopoiesis is closest for online policy synthesis, more ambitious than our bounded screen (Section 5), but likewise omits this formulation.

Methodologically, SBS/VBS envelopes follow classical algorithm selection and portfolios [26, 8, 37] as specialized in SATzilla, AutoFolio, and Hydra [17, 36]; we claim no novelty for those concepts, nor for decision-critical states [14] or counterfactual action-value estimation [19] in sequential decision-making generally, neither of which targets LLM-serving scheduling. Serving differs: actions change future queue and resource state, so regime accuracy need not imply action accuracy—a closed-loop covariate-shift motif familiar from imitation learning [28]. Our contribution is a causal terminal-utility (counterfactual-ANWG) characterization of such consequential decisions specifically within LLM-serving scheduler portfolios, showing that native disagreement and a short-horizon completion proxy are unreliable surrogates for it here. Typed synthesis inherits GP and hyper-heuristic precedents [15, 21, 34, 3]; our screen differs mainly by exact parent reproduction, a portfolio-envelope objective, and pre-specified nonredundancy criteria rather than by inventing evolutionary search [27, 13].

Limitations. Workloads are synthetic; one public replay saturates under our objective, and even a stressed public construction did not separate P_6/VTC rankings under the tested setup. The six policies plus disclosed external proxies are mechanism-diverse baselines, not an exhaustive SOTA leaderboard; closely related systems (H-MAS, Libra, SOLA, SLOs-Serve, Autopoiesis) are discussed but not reproduced, since integrating them exceeds adding one P_6 policy, and P_{ext} was not used to retrain frozen adaptive experiments. Real-system evidence uses a single vLLM version, model, GPU, and mechanism family. The terminal-ANWG study (Section 4.3) is scoped to TRAIN/VAL A/B/C, not joint-240, with only 734 acquired branches (27 nonzero) under fixed caps and family imbalance, so scenario-level concentration estimates carry wide bootstrap intervals. The joint-240 same-distribution result (Section 4.2) evaluates only the two preregistered causal architectures tested, not all possible adaptive schedulers; this negative result is evidence against those architectures and proxies here, not an impossibility claim. Finally, ANWG reflects a chosen weighted SLO-goodput objective.

Conclusion. LLM-serving portfolios can exhibit measurable complementarity and positive VBS headroom—including after expanding P_6 with independently motivated external baselines. On the same 240-scenario joint workload used to measure that headroom, neither evaluated causal adaptive method recovered it: both fell below the single best fixed scheduler, with bootstrap CIs excluding zero improvement (Table 4). A separate causal terminal-utility analysis offers one explanation: consequential scheduling decisions are sparse and concentrated, and both native disagreement and an earlier short-horizon proxy are unreliable guides to them (Table 3). This grounds the opportunity–exploitability distinction in a genuine same-distribution comparison rather than an unresolved gap. Future work should test adaptive architectures informed by this concentration structure and continue validating simulator mechanisms against engine semantics before transfer claims.

Acknowledgments. The author thanks Professor Ioannis Koutis for guidance and support, Anders Borum for lifetime access to Secure ShellFish supporting the research workflow, and the author's mother for continued support.

Data and Code Availability Code, workload generators, configurations, and derived artifacts will be made publicly available at `https://github.com/SoroushVahidi/llm-serving-heuristic-evolution` upon publication. Simulator results use project-generated synthetic workloads and, for public-trace replay, derived windows from BurstGPT and Azure 2023 traces [33, 20, 23].

Generative AI Use The author used ChatGPT, Claude, Codex, and Perplexity AI for organization, language refinement, literature-search planning, software-development tasks, and workflow support. The author verified the designs, results, interpretations, citations, and final content, and takes responsibility for the work.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B.S., Tumanov, A., Ramjee, R.: Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 117–134. USENIX Association (2024)
2. Ao, R., Luo, G., Simchi-Levi, D., Wang, X.: Optimizing LLM inference: Fluid-guided online scheduling with memory constraints. arXiv preprint arXiv:2504.11320 (2025)
3. Branke, J., Hildebrandt, T., Scholz-Reiter, B.: Hyper-heuristic evolution of dispatching rules: A comparison of rule representations. Evolutionary Computation **23**(2), 249–277 (2015)
4. Chen, S., Jia, Z., Khan, S., Krishnamurthy, A., Gibbons, P.B.: SLOs-Serve: Optimized serving of multi-SLO LLMs (2025), preprint
5. Cho, E., Bang, J., Hwang, R., Rhu, M.: PASCAL: A phase-aware scheduling algorithm for serving reasoning-based large language models. In: Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE (2026). https://doi.org/10.1109/HPCA68181.2026.11408493, also available as arXiv:2602.11530
6. Demers, A., Keshav, S., Shenker, S.: Analysis and simulation of a fair queueing algorithm. In: Proceedings of the ACM Symposium on Communications Architectures and Protocols (SIGCOMM). pp. 1–12. ACM (1989)
7. Fu, Y., Zhu, S., Su, R., Qiao, A., Stoica, I., Zhang, H.: Efficient LLM scheduling by learning to rank. In: Advances in Neural Information Processing Systems 37 (2024)
8. Gomes, C.P., Selman, B.: Algorithm portfolios. Artificial Intelligence **126**(1–2), 43–62 (2001)
9. Gong, R., Bai, S., Wu, S., Fan, Y., Wang, Z., Li, X., Yang, H., Liu, X.: Past-future scheduler for LLM serving under SLA guarantees. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25). pp. 798–813. ACM (2025). https://doi.org/10.1145/3676641.3716011
10. Hong, K., Li, X., Chen, L., Mao, Q., Dai, G., Ning, X., Yan, S., Liang, Y., Wang, Y.: SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling. In: Proceedings of Machine Learning and Systems. vol. 7 (2025)

11. Hu, C., Huang, H., Xu, L., Chen, X., Xu, J., Chen, S., Feng, H., Wang, C., Wang, S., Bao, Y., Sun, N., Shan, Y.: Inference without interference: Disaggregate LLM inference for mixed downstream workloads (2024), preprint; system name TetriInfer
12. Jaillet, P., Jiang, J., Podimata, C., Zhou, Z.: Online scheduling for LLM inference with KV cache constraints. arXiv preprint arXiv:2502.07115 (2025)
13. Jiang, Y., Yan, R., Peng, Y., Li, W., Wang, T., Fu, F., Yuan, B.: Autopoiesis: A self-evolving system paradigm for LLM serving under runtime dynamics. arXiv preprint arXiv:2604.07144 (2026)
14. Karino, I., Ohmura, Y., Kuniyoshi, Y.: Identifying critical states by the action-based variance of expected return. In: Artificial Neural Networks and Machine Learning – ICANN 2020, Lecture Notes in Computer Science, vol. 12397, pp. 366–378. Springer (2020). https://doi.org/10.1007/978-3-030-61609-0_29
15. Koza, J.R.: Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press, Cambridge, MA (1992)
16. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with PagedAttention. In: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023). pp. 611–626. ACM (2023)
17. Lindauer, M., Hoos, H.H., Hutter, F., Schaub, T.: AutoFolio: An automatically configured algorithm selector. Journal of Artificial Intelligence Research **53**, 745–778 (2015)
18. Liu, Y., Xu, C., Jia, Q., Wang, Y., Chen, F., Jin, L., Liu, L., Zhao, Y., Ding, Y., Li, X.: H-MAS: Hierarchical multi-agent scheduling for multi-tenant LLM serving. In: Findings of the Association for Computational Linguistics: ACL 2026. pp. 39051–39071. Association for Computational Linguistics (2026)
19. Mesnard, T., Weber, T., Viola, F., Thakoor, S., Saade, A., Harutyunyan, A., Dabney, W., Stepleton, T.S., Heess, N., Guez, A., Moulines, E., Hutter, M., Buesing, L., Munos, R.: Counterfactual credit assignment in model-free reinforcement learning. In: Proceedings of the 38th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 139, pp. 7654–7664. PMLR (2021)
20. Microsoft Azure: Azure LLM inference trace 2023. `https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md` (2023), public trace dataset accompanying Splitwise
21. Montana, D.J.: Strongly typed genetic programming. Evolutionary Computation **3**(2), 199–230 (1995)
22. Parekh, A.K., Gallager, R.G.: A generalized processor sharing approach to flow control in integrated services networks: The single-node case. IEEE/ACM Transactions on Networking **1**(3), 344–357 (1993)
23. Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, I., Maleki, S., Bianchini, R.: Splitwise: Efficient generative LLM inference using phase splitting. In: Proceedings of the 51st Annual International Symposium on Computer Architecture. IEEE Computer Society (2024). https://doi.org/10.1109/ISCA59077.2024.00019
24. Patke, A., Reddy, D., Jha, S., Qiu, H., Pinto, C., Narayanaswami, C., Kalbarczyk, Z.T., Iyer, R.K.: Queue management for SLO-oriented large language model serving. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC 2024). pp. 18–35. ACM (2024). https://doi.org/10.1145/3698038.3698523
25. Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., Xu, X.: Mooncake: Trading more storage for less computation – a KVCache-centric architecture for serving LLM chatbot. In: 23rd USENIX Conference on File and Storage Technologies (FAST 25). pp. 155–170. USENIX Association (2025)

26. Rice, J.R.: The algorithm selection problem. Advances in Computers **15**, 65–118 (1976)
27. Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J.R., Ellenberg, J.S., Wang, P., Fawzi, O., Kohli, P., Fawzi, A.: Mathematical discoveries from program search with large language models. Nature **625**(7995), 468–475 (2024)
28. Ross, S., Gordon, G.J., Bagnell, J.A.: A reduction of imitation learning and structured prediction to no-regret online learning. In: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 15, pp. 627–635. PMLR (2011)
29. Ruan, C., Chen, Y., Tian, D., Shi, Y., Wu, Y., Li, J., Li, C.: Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads. In: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). USENIX Association (2026)
30. Schrage, L.: A proof of the optimality of the shortest remaining processing time discipline. Operations Research **16**(3), 687–690 (1968)
31. Sheng, Y., Cao, S., Li, D., Zhu, B., Li, Z., Zhuo, D., Gonzalez, J.E., Stoica, I.: Fairness in serving large language models. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 965–988. USENIX Association (2024)
32. Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., Lin, W.: Llumnix: Dynamic scheduling for large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 173–191. USENIX Association (2024)
33. Wang, Y., Chen, Y., Li, Z., Kang, X., Fang, Y., Zhou, Y., Zheng, Y., Tang, Z., He, X., Guo, R., Wang, X., Wang, Q., Zhou, A.C., Chu, X.: BurstGPT: A real-world workload dataset to optimize LLM serving systems. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 5831–5841. ACM (2025). https://doi.org/10.1145/3711896.3737413
34. Whigham, P.A.: Grammatically-based genetic programming. In: Proceedings of the Workshop on Genetic Programming: From Theory to Real-World Applications. pp. 33–41 (1995)
35. Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., Jin, X.: FastServe: Iteration-Level preemptive scheduling for large language model inference. In: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26). pp. 57–74. USENIX Association (2026)
36. Xu, L., Hoos, H.H., Leyton-Brown, K.: Hydra: Automatically configuring algorithms for portfolio-based selection. In: Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence. pp. 210–216 (2010)
37. Xu, L., Hutter, F., Hoos, H.H., Leyton-Brown, K.: SATzilla: Portfolio-based algorithm selection for SAT. Journal of Artificial Intelligence Research **32**, 565–606 (2008)
38. Yu, G.I., Jeong, J.S., Kim, G.W., Kim, S., Chun, B.G.: Orca: A distributed serving system for transformer-based generative models. In: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). pp. 521–538. USENIX Association (2022)
39. Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H.: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). pp. 193–210. USENIX Association (2024)